RfCardDevice

Prepare

```java
import com.pos.sdk.DeviceManager;
import com.pos.sdk.DevicesFactory;
import com.pos.sdk.callback.ResultCallback;
import com.pos.sdk.rfcard.RfCardDevice;
import ...

    private RfCardDevice mRfCardDevice;
    private Context context;

    DevicesFactory.create(context, new ResultCallback() {
        @Override
        public void onFinish(DeviceManager deviceManager) {
            mRfCardDevice = deviceManager.getRfDevice();
        }

        @Override
        public void onError(int i, String s) {
        }
    });
```

Interface

Reset

Contactless Card Reset

```java
    /**
     * To reset the RF card to its original state
     *
     * @return reset result
     *         If success, return byte[]
     *         If fail, return null
     */
    public byte[] reset()
```

ResetCard

```java
    /**
     * To reset a specify card type. Actually, this api is only use for reset special card.
     * In daily payment application , reset() is enough.
     *
     * @param cardType
     *        0x80:TYPE A
     *        0x40:TYPE B
     *        0x20:TYPE C(included Felica)
     *        0xC0:TYPE A + TYPE B
     *        0xA0:TYPE A + TYPE C
     *        0x60:TYPE B + TYPE C
     *
     * @return reset result
     *         If success, return byte[]
     *         If fail, return null
     */
    public byte[] resetCard(byte cardType)
```

512Word

Exists

Detecting whether the card has been placed Contactless

```java
/**
     * Check whether the RF card is detected
     *
     * @return card status
     *         Ture:  the RF card exists
     *         False: the RF card doesn't exist
     */
    public boolean exists()
```

ReadCardType

Get the card type

```java
/**
     * To read the type of the RF card
     * @return type
     *         1:  ISO14443_A
     *         2:  ISO14443_B
     *         4:  ISO15693
     *         8:  ICODE1
     *         16: MIFARE1
     */
    public int readCardType()
```

Send

APDU interaction

```java
/**
     * To send the APDU command
     * @param APDU command
     * @return byte[] Response from the card
     */
    public byte[] send(byte[] APDU)
```

Halt

Interrupt contactless card

```java
/**
     * To halt the RF device
     * @return halt result
     *         if return byte[] , means halt success
     *         if return null, means halt fail
     */
    public byte[] halt()
```

Example

Java

Java | Run Code | Copy

```java
private void onReset() {
    byte[] result = mRfCardDevice.reset();
    showNormalMessage("Reset result = " + Arrays.toString(result));
}

private void onResetCard() {
    byte[] result = mRfCardDevice.resetCard((byte)0x40);
    showNormalMessage("Reset card result = "+ Arrays.toString(result));
}

private void onReadCardType() {
    int result = mRfCardDevice.readCardType();
    showNormalMessage("Card Type = " + result);
}

private void onSend() {
    byte[] result = mRfCardDevice.send(hexStringToByte("00A404000E315041592E5359532E4444463031"));
    showNormalMessage("Send result = " + Arrays.toString(result));
}

private void onStatus() {
    boolean result = mRfCardDevice.exists();
    showNormalMessage("Status = " + result);
}

private void onHalt() {
    byte[] result = mRfCardDevice.halt();
    showNormalMessage("Halt result = " + Arrays.toString(result));
}

private void onSetFelicaParam() {
    boolean result = mRfCardDevice.setFelicaParam(0, new byte[]{0x2C,0x01});
    showNormalMessage("SetFelicaParam result = " + result);
}
```

Kotlin

Kotlin | Copy

```kotlin
    private fun onReset() {
        val result = mRfCardDevice!!.reset()
        showNormalMessage("Reset result = " + Arrays.toString(result))
    }

    private fun onResetCard() {
        val result = mRfCardDevice!!.resetCard(0x40.toByte())
        showNormalMessage("Reset card result = " + Arrays.toString(result))
    }

    private fun onReadCardType() {
        val result = mRfCardDevice!!.readCardType()
        showNormalMessage("Card Type = $result")
    }

    private fun onSend() {
        val result = mRfCardDevice!!.send(hexStringToByte("00A404000E315041592E5359532E4444463031"))
        showNormalMessage("Send result = " + Arrays.toString(result))
    }

    private fun onExists() {
        val result = mRfCardDevice!!.exists()
        showNormalMessage("Is existed = $result")
    }

    private fun onHalt() {
        val result = mRfCardDevice!!.halt()
        showNormalMessage("Halt result = " + Arrays.toString(result))
    }

    private fun onSetFelicaParam() {
        val result = mRfCardDevice!!.setFelicaParam(0, byteArrayOf(0x2C, 0x01))
        showNormalMessage("SetFelicaParam result = $result")
    }
```

Demo

RfCardFragment.java (4 KB)